

CISC 719-90-O/CISC 619-90-O-2026**Name: Stellamaris Nakacwa****Instructors: Professor Majid Shaalan & Professor Mike Shahine****Assignment: IMC05 - The Sieve of Eratosthenes****Part 1: Classical + Parallel implementation**

I implemented and benchmarked multiple versions of the Sieve of Eratosthenes under progressively more sophisticated memory and parallel models. My primary focus was on handling values of $N \geq 10^9$, where memory efficiency becomes the dominant constraint.

I implemented the following execution models:

- OpenMP (shared memory)
- MPI (distributed memory)
- CUDA GPU
- Hybrid MPI + OpenMP

Before scaling to very large N , I deliberately implemented the sieve in stages to understand memory behavior:

Assumption:

Approach	Approx Memory ($N=10^9$)	Feasible?
Naive byte array	~1 GB	Yes
Odd-only byte	~0.5 GB	Yes
Bit-packed	~62 MB	Very safe
Segmented bit-packed	Configurable (e.g., 32–64MB)	Best

Applying all optimizations at once initially obscured how each memory compression step impacted feasibility and performance. By separating naive, odd-only, bit-packed, and segmented implementations for both OpenMP and MPI, I was able to directly observe how memory complexity reduced from $O(N)$ to $O(S)$, where S is segment size. Segmentation was essential for scaling beyond 10^9 . Without it, even bit-packed versions eventually become impractical for very large N .

1.1. PCAM Application:

I explicitly applied the PCAM methodology throughout:

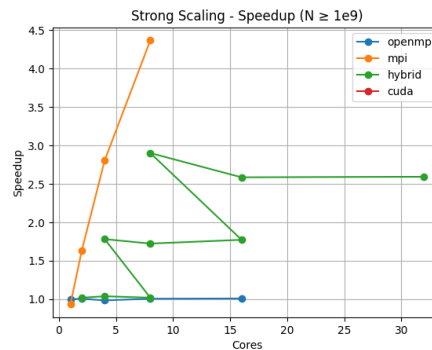
- Partitioning:
 - OpenMP: partitioned segments across threads.
 - MPI: partitioned global odd index space across ranks.
 - CUDA: partitioned primes across threads within segments.
- Communication:
 - MPI: broadcast base primes and reduced counts.
 - CUDA: host–device memory transfers per segment.
- Agglomeration:
 - Segment size-controlled task granularity.
 - Block sizes-controlled CUDA tiling.
- Mapping:
 - OpenMP mapped segments to threads dynamically.
 - MPI mapped contiguous index blocks to ranks.
 - Hybrid mapped segments to OpenMP threads inside MPI ranks.

Breaking the problem apart using PCAM made the mapping between algorithm structure and hardware structure much clearer.

Part 2: Comparative Analysis

All benchmarking focused on segmented bit-packed versions for $N \geq 10^9$.

1. Speedup vs Single-Thread Baseline

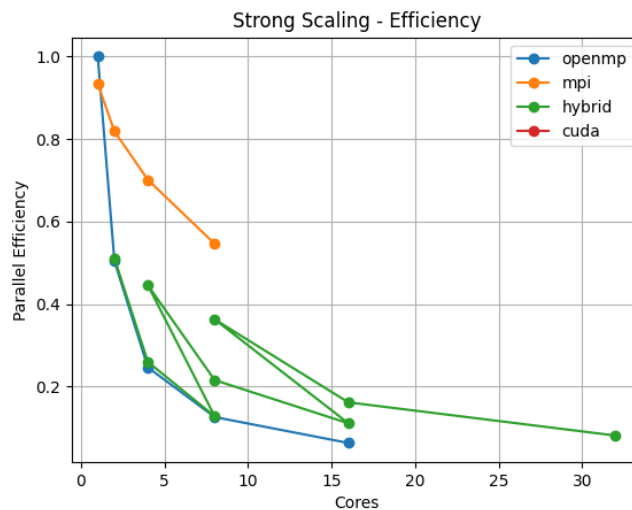


Using the 1-thread OpenMP run as baseline:

- MPI achieved the strongest speedup ($\approx 4.4\times$ at 8 ranks).
- OpenMP showed minimal speedup beyond 2 threads.
- Hybrid plateaued around $2\text{--}2.6\times$.
- CUDA provided a consistent speedup over the 1-core baseline but was limited by host–device transfers.

The key observation is that this workload is memory-bandwidth bound, not compute-bound. OpenMP threads contend with shared memory bandwidth, limiting scalability. MPI performs better because each rank operates in an isolated memory space.

2. Strong Scaling

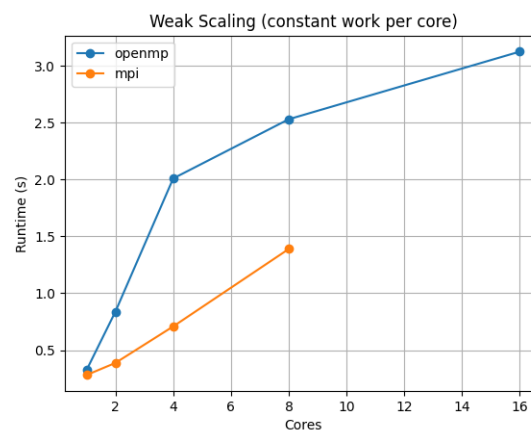


For fixed $N = 10^9$:

- MPI scaled most effectively.
- OpenMP efficiency dropped rapidly.
- Hybrid did not outperform pure MPI on a single node.

This confirms that distributed memory reduces contention and improves scalability for memory-intensive algorithms.

3. Weak Scaling

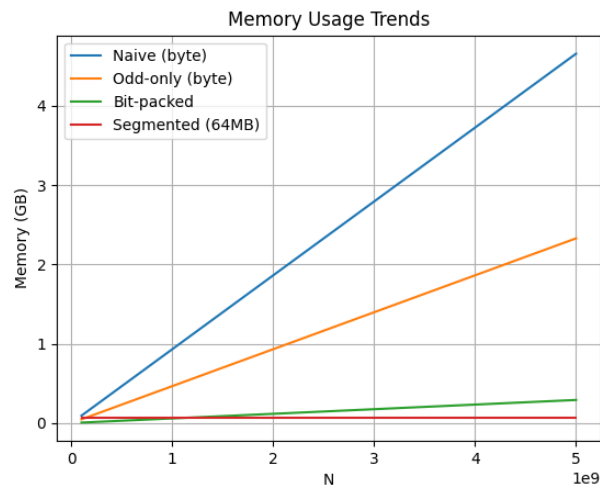


Under constant work per core:

- MPI runtime remained nearly flat.
- OpenMP runtime increased steadily.

This indicates that shared-memory contention becomes more severe as threads increase, whereas distributed memory maintains locality.

4. Memory Usage Trends



The memory plot clearly demonstrated:

- Naive and odd-only grow linearly with N.
- Bit-packed reduces memory by 16×.
- Segmented remains constant.

This result strongly validates segmentation as the enabling technique for large-scale sieving.

Part 2.1: Qtns

1. When Each Model Is Preferable:

OpenMP

- Best for moderate problem sizes on a single multi-core node.
- Simple to implement.
- Limited by shared memory bandwidth.

MPI

- Preferable for large N on multi-node systems.
- Strong scalability.
- Better memory locality per rank.

Hybrid MPI+OpenMP

- Ideal for multi-socket clusters.
- Can balance inter-node and intra-node parallelism.
- On a single node, may not outperform pure MPI.

CUDA GPU

- Preferable when computation per memory access is high.
- Limited by atomic operations and host–device transfer overhead.
- Strong potential if reduction is fully performed on-device.

2. *Hardware Characteristics That Most Affect Performance:*

1. Memory bandwidth (dominant factor)
2. Cache size and hierarchy
3. NUMA layout
4. Interconnect latency (MPI)
5. GPU global memory throughput
6. PCIe bandwidth

Because sieve marking is simple arithmetic but heavy memory writes, performance scales primarily with bandwidth rather than FLOPS.

3. *Bottlenecks as N Grows. As N increases:*

- Naive implementations become infeasible due to memory exhaustion.
- Bit-packed versions become memory-bandwidth bound.
- MPI reduction cost grows with rank count.
- CUDA atomic contention increases with segment density.
- Host-device transfers become visible bottlenecks.

The most fundamental bottleneck is memory traffic.

Part 3: Real-World Extension

I extended the sieve concept into a 2D image domain using CUDA.

I implemented:

- `cuda_prime_mask` — modulo-based prime grid masking
- `cudawindowfilter` — prime-intensity window convolution filter

PERSONAL THOUGHTS:

1. *What was easy vs hard in adapting the classical sieve to a real-world domain?*

Mapping the sieve to image space conceptually was straightforward: primes were interpreted as periodic sampling intervals or radial intensity windows.

The difficulty was not conceptual but structural. Applying all models simultaneously under the PCAM framework was initially overwhelming. Separating memory optimization steps first allowed me to understand CPU memory behavior before mapping to GPU kernels.

The prime modulo mask was easier, since it required only coordinate checks. The prime-intensity window filter was harder because it required designing a meaningful convolution kernel

shaped by prime radii.

2. *How does the PCAM model help or hinder thinking creatively?*

PCAM helped significantly in organizing my thinking. It forced me to explicitly consider:

- What is independent work?
- What data must be shared?
- What is the communication cost?
- How does this map to hardware topology?

However, applying PCAM to every model at once initially hindered creativity. Breaking the problem into progressive steps made it manageable.

3. *What would you do differently if you had access to a distributed GPU cluster?*

If I had access to a distributed GPU cluster:

- I would combine MPI across nodes with CUDA per node.
- Base primes would be broadcast once.
- Each node would process independent segments on its GPU.
- Final reductions would occur via MPI.

I would also:

- Move pop-count reduction entirely to GPU.
- Use CUDA shared memory tiling for convolution.
- Explore multi-GPU per node scaling.

This would likely produce near-linear scalability for very large N.

CONCLUSION:

- ✓ Segmentation and bit-packing were more impactful than increasing core count.
- ✓ The PCAM model, when applied progressively, provided a clear framework to reason about computation, communication, and scalability across heterogeneous architectures.